Architecture Design Document: Multi-Agent Clinical Triage System

1. System Overview & Objective

This document defines the technical architecture for a decentralized, multi-agent clinical decision support system built on **Spring AI 2.0**. The system demonstrates interoperability between two emerging AI interaction standards:

- **Agent-to-Agent (A2A) Protocol:** Governs autonomous, high-level task delegation and intent exchange between independent domain agents.
- **Model Context Protocol (MCP):** Standardizes how agents securely expose and consume deterministic backend tools, patient records, and domain systems.

The solution operates fully on-premises without external cloud dependencies. Local model inference is managed by **Ollama**, ensuring patient health information (PHI) remains strictly contained within local network boundaries.

2. Architectural Principles & Protocol Boundary

INTERACTION TAXONOMY	
Agent-to-Agent (A2A) Layer	Model Context Protocol (MCP) Layer
• Autonomous coordination • Natural-language task delegation • Peer-to-peer discovery (Agent Card) • State/Task negotiation • Agent ↔ Agent	• Deterministic tool invocation • Structured schema execution • System-to-system integration • Read/Write data access • Agent ↔ Backend / EHR Database

The system segregates **reasoning/delegation** from **context/data access**:

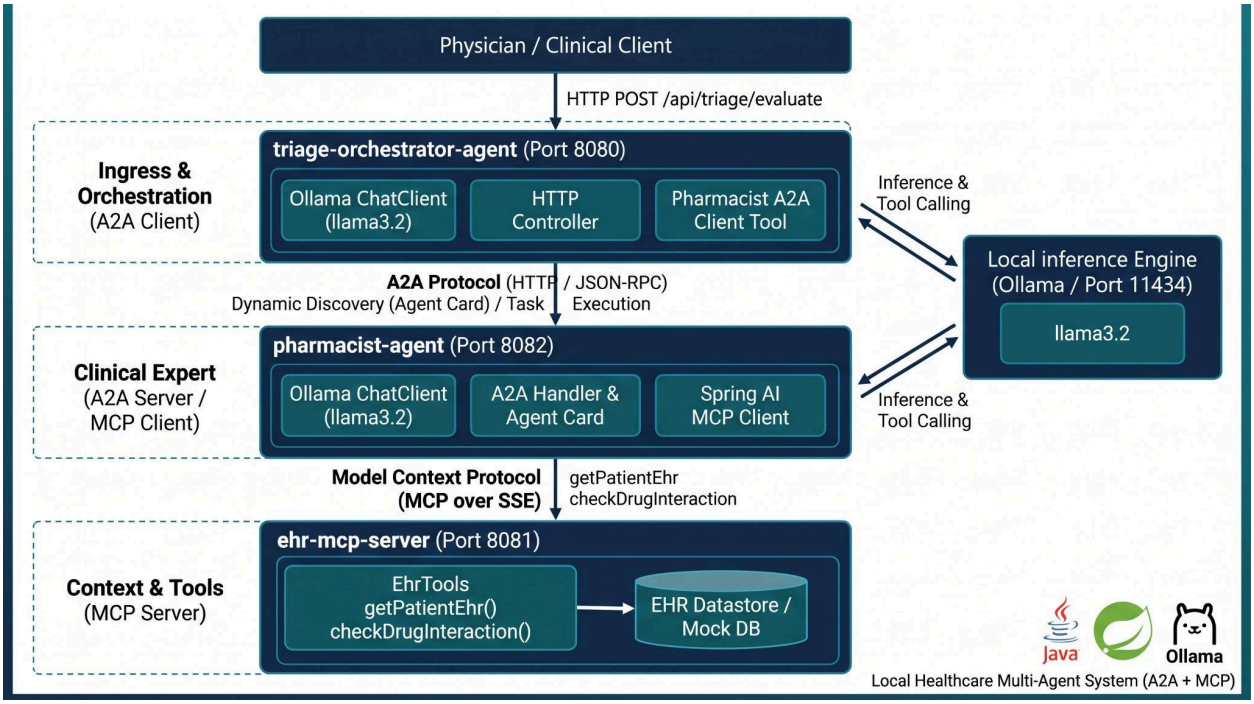
1. **Delegation via A2A:** When an agent lacks domain expertise (e.g., triage vs. pharmacology), it delegates sub-tasks across process boundaries using standardized JSON-RPC schemas rather than hardcoded point-to-point business logic.
2. **Context Resolution via MCP:** When an agent requires factual enterprise data, it does not query databases directly. It connects as an MCP Client to an MCP Server exposing schema-backed tool interfaces over Server-Sent Events (SSE).

3. System Topology & Component Layout

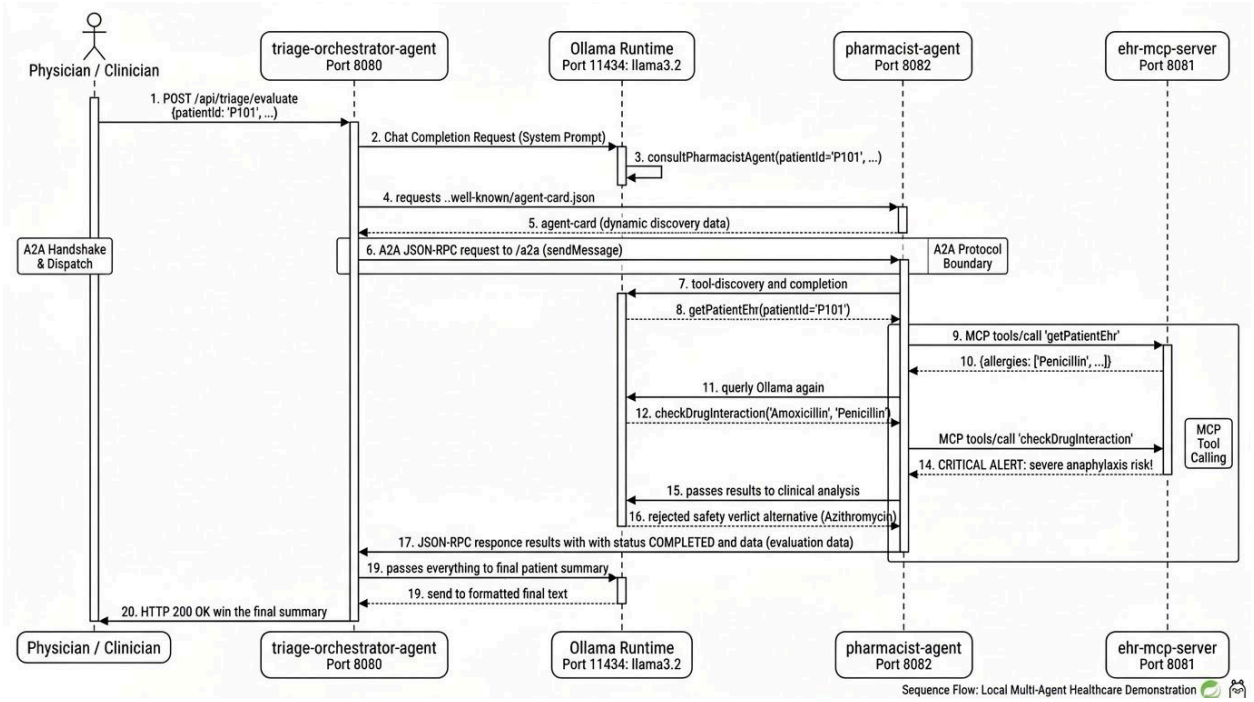
The deployment topology consists of three isolated Spring Boot applications and an Ollama inference daemon running locally:

Component	Role	Runtime Port	Primary Protocols	Responsibilities

triage-orchestrator-agent	Ingress Gateway & Master Orchestrator	8080	HTTP / REST, A2A Client	Ingests physician queries, plans workflow, executes dynamic A2A discovery, synthesizes final clinical verdict.
pharmacist-agent	Specialist Autonomous Agent	8082	A2A Server, MCP Client	Publishes Agent Card, consumes A2A tasks, directs LLM reasoning to call MCP tools, assesses contraindications.
ehr-mcp-server	Resource & Tool Provider	8081	MCP Server (SSE)	Serves Electronic Health Records (EHR) and rules-based drug-allergy interaction engines via MCP.
Ollama Daemon	Local Neural Inference Engine	11434	HTTP / REST	Hosts llama3.2 for structured tool calling, reasoning, and textual generation.



4. Sequence Flow Diagram



5. Interface Specifications & Contracts

5.1 Ingress Contract (Physician to Triage Gateway)

- **Method / Path:** POST /api/triage/evaluate
- **Content-Type:** application/json

JSON

```
{
  "patientId": "P101",
  "diagnosis": "Acute Bacterial Pharyngitis",
  "proposedPrescription": "Amoxicillin 500mg TID"
}
```

5.2 Agent-to-Agent (A2A) Protocol Contracts

A. Discovery: Agent Card

- **Endpoint:** GET /.well-known/agent-card.json
- **Host:** http://localhost:8082

JSON

```
{
  "name": "pharmacist-agent",
  "description": "Specialist Agent for drug-safety, allergy contraindication, and dosage verification.",
  "version": "1.0.0",
  "skills": [
    {
      "id": "prescribe-safety-check",
      "description": "Evaluates patient EHR and cross-checks proposed drug safety and interactions."
    }
  ],
  "endpoints": {
    "rpc": "http://localhost:8082/a2a"
  }
}
```

B. Execution: JSON-RPC Task Request

- **Endpoint:** POST /a2a
- **Host:** http://localhost:8082

```
JSON
{
  "jsonrpc": "2.0",
  "method": "sendMessage",
  "params": {
    "query": "Validate safety for patient 'P101'. Proposed Rx: 'Amoxicillin 500mg TID'. Diagnosis: 'Acute Bacterial Pharyngitis'."
  },
  "id": "550e8400-e29b-41d4-a716-446655440000"
```

C. Execution: JSON-RPC Task Response

```
JSON
{
  "jsonrpc": "2.0",
  "result": {
    "status": "COMPLETED",
    "artifact": "SAFETY ASSESSMENT: REJECTED.\nPatient P101 has a documented Penicillin allergy. Amoxicillin presents severe risk of anaphylaxis. Recommended alternative: Azithromycin 500mg."
  },
  "id": "550e8400-e29b-41d4-a716-446655440000"
}
```

5.3 Model Context Protocol (MCP) Interface

The ehr-mcp-server publishes tools conforming to the MCP Tools Specification over an SSE transport channel (http://localhost:8081/sse):

```
JSON
[
  {
    "name": "getPatientEhr",
    "description": "Fetch patient Electronic Health Record (EHR) including known drug allergies, active diagnoses, and current medications.",
    "inputSchema": {
      "type": "object",
      "properties": {
        "patientId": {
          "type": "string",
          "description": "The unique patient ID, e.g. P101"
        }
      },
      "required": ["patientId"]
```

```
    }
   },
   {
    "name": "checkDrugInteraction",
    "description": "Check for contraindications between a proposed medication and a patient's existing medication or condition.",
    "inputSchema": {
     "type": "object",
     "properties": {
      "proposedDrug": {
       "type": "string",
       "description": "Proposed medication name"
      },
      "existingEntity": {
       "type": "string",
       "description": "Existing medication or allergy class"
      }
     },
     "required": ["proposedDrug", "existingEntity"]
    }
   }
]
```

6. Component-Level Design Details

6.1 triage-orchestrator-agent (Port 8080)

- **Role:** Central gateway. Does not have direct access to database records or specialized pharmacology rules.
- **LLM Configuration:** Temperature set to 0.2 to minimize hallucination and prioritize deterministic workflow routing.
- **Dynamic Tool Registration:** Registers PharmacistA2AClientTool via @Tool. When the orchestrator LLM recognizes a prescription instruction, it invokes this tool. The tool performs runtime discovery via the target's Agent Card URL, then constructs and dispatches the A2A JSON-RPC frame.

6.2 pharmacist-agent (Port 8082)

- **Role:** Clinical domain specialist.
- **Dual Adapter Pattern:**
  - Exposes an **inbound HTTP adapter** (PharmacistA2AController) adhering to the A2A spec.
  - Implements an **outbound MCP client** bound through Spring AI's ToolCallbackProvider.
- **Dynamic Context Injection:** Discovered MCP tools are passed directly into the ChatClient builder (.defaultTools(mcpToolCallbackProvider.getToolCallbacks())), letting

the LLM query patient records on demand.

6.3 ehr-mcp-server (Port 8081)

- **Role:** Pure backend data/logic provider.
- **Decoupled Architecture:** Operates without an LLM dependency. Tools are defined using Spring AI's @Tool annotations on standard Spring services and served via spring-ai-starter-mcp-server-webmvc.
- **Stateless Isolation:** Serves requests strictly as an MCP server. It maintains internal consistency and isolates PHI from direct exposure to outside agent processes.

7. Healthcare Governance & Operational Safety

1. **Air-Gapped PHI Boundary:** Using Ollama locally guarantees that clinical notes, patient identification numbers (P101), and medical histories never leave the hosting machine, satisfying HIPAA/HITECH data privacy requirements.
2. **Deterministic Fallbacks:** The MCP server contains deterministic logic for critical alerts (e.g., cross-checking beta-lactam classes). This ensures that life-safety rules do not rely solely on generative LLM probabilities.
3. **Auditability & Traceability:** The separation between A2A layers and MCP layers enables discrete logging:
   - **A2A Audit Logs:** Capture clinical reasoning and inter-departmental agent consultations.
   - **MCP Audit Logs:** Capture exact records accessed and deterministic rules executed for medical compliance verification.